

Workflow and Used Approaches

Terraform :

For creating the cloud infrastructure, I used Terraform. The main reason I decided to use Terraform is because it allows me to define all the infrastructure in code format. So instead of manually creating resources in AWS console again and again, I can just run Terraform and the same infrastructure will be created automatically. This makes the setup repeatable and reliable. To make the project easier to manage, I divided my Terraform code into separate modules.

Modules:

VPC Module -> This module is mainly for creating the network setup. Here I created the VPC, public and private subnets, route tables, and also the internet gateway. Basically all the networking related resources are defined here. The VPC acts like the main network where all other resources will run.

EKS Module -> In this module I created the EKS Kubernetes cluster. It also creates the node groups which are basically the worker machines where the containers will run. Using EKS helped because AWS manages the Kubernetes control plane.

EC2 Module -> Here I created a Bastion Host. Since the EKS cluster is inside a private subnet, we cannot directly access it from outside. So this EC2 instance acts like a secure entry point. I can SSM into this server and run commands like kubectl to manage the Kubernetes cluster.

IAM Module -> This module is responsible for creating IAM roles and policies. These permissions allow EKS to communicate with other AWS services. For example, the cluster needs permission to pull container images from ECR .

Security Groups Module

In this module I defined the firewall rules. Security groups control which traffic is allowed to enter or leave the resources. For example, allowing SSH to the bastion host or allowing Kubernetes nodes to communicate internally.

Why Modules : Dividing everything into modules makes the code more reusable and organized. For example if I want to create another environment later, I can reuse the same VPC module without rewriting the code again. Another thing I did was configure the Terraform state backend in Amazon S3. Terraform keeps track of infrastructure using a state file. If that file is only on my laptop and something happens to it, the infrastructure tracking will break. So storing it in S3 makes it safer .

Helm Chart Layout:

For deploying my Python microservices to Kubernetes, I used Helm. Helm basically helps package Kubernetes manifests in a reusable way.

In my project there are four Python services, and all of them follow the same Helm chart structure.

Helm Chart Files

- Chart.yaml -> This file contains the basic information about the Helm chart like name and version.
- values.yaml -> This file is very useful because I can define configurable values here like:
- Docker image tag
- Service ports
- Replica count
- Instead of editing the deployment YAML every time, I just update the values in this file.
- templates folder

This folder contains the Kubernetes resource templates such as:

- Deployment
- Service

These templates use variables from values.yaml, which makes easier

Namespace and Access Model

In Kubernetes, putting everything in the default namespace is not a best practice.

So I decided to create separate namespaces to keep things isolated.

Namespaces I Created

- frontend -> This namespace contains the UI application that users interact with.
- backend-users -> This namespace handles user related services.
- backend-products -> This namespace manages product related APIs.
- backend-orders -> This namespace handles order processing.
- database -> This namespace is dedicated for the Percona database cluster.
- api-gateway -> This namespace contains the Kong API Gateway, which acts like the entry point for external traffic.

Cross Namespace Communication

- Namespaces provide isolation, which is good for security, but it also means services cannot freely talk to each other.
- To solve this, I used Gateway API ReferenceGrants. This allows Kong API Gateway to access services running in other namespaces.
- I also implemented Network Policies. By default, I created a deny-all rule so that no pod can talk to another pod unless I specifically allow it.
- For example, only the orders service is allowed to connect to the database on port 5432. Other services cannot access it.

This way the communication inside the cluster is strictly controlled.

End-to-End CI/CD Flow

The CI/CD pipeline is probably my favorite part of this project because everything is fully automated.

Step 1 – Code Push

Whenever I push my code to GitHub, the pipeline automatically starts.

Step 2 – GitHub Actions (CI)

GitHub Actions runs the CI pipeline.

- First it builds the Docker image of the application.
- After building the image, it runs a Trivy security scan. This step is important because it checks if the container image contains any vulnerabilities.
- If Trivy finds any high severity issues, the pipeline fails immediately. This ensures insecure images are not deployed.
- Step 3 – Push Image to ECR
- If the scan passes successfully, the Docker image is pushed to Amazon ECR.

The image is tagged with the Git commit SHA, which helps track exactly which version of the code is deployed.

Step 4 – GitOps Deployment

After pushing the image, the CI pipeline updates the values.yaml file in the CD repository with the new image tag.

Step 5 – ArgoCD Deployment

- ArgoCD continuously watches the Git repository.
- Whenever it detects a change in the Helm configuration, it automatically syncs the Kubernetes cluster with the latest configuration.

Security Enforcement

Since the goal was to make the system production-grade, I added multiple layers of security.

Kyverno (Admission Controller)

Kyverno acts like a policy gatekeeper for Kubernetes.

For example:

- If a container tries to run as root user, Kyverno blocks it
- If a pod does not define CPU or memory limits, the deployment will be rejected.

This ensures all deployments follow security best practices.

Network Policies

I implemented a default deny network policy.

This means that by default, no pod can communicate with another pod unless I explicitly create an allow rule.

For example:

Only the orders service is allowed to connect to the database on port 5432. Other services cannot access the database.

This approach improves internal cluster security.

Sealed Secrets

- One important rule and best practice is never store plain secrets in Git.
- So instead of storing passwords directly, I used Sealed Secrets.
- First I encrypt the secret using kubeseal. The encrypted version is stored in Git.
- Only the Sealed Secrets controller inside the Kubernetes cluster can decrypt it and create the actual Kubernetes secret.
- This way the secrets remain secure while still following the GitOps workflow.